MIDTERM LABORATORY EXAMINATION

Introduction to Computer Graphics and Visual Computing

Interactive and Creative C++ OpenGL/GLUT Graphics Application

Course: CS0045 – Computer Graphics and Visual Computing

Companion: Module 1 to 4

Toolchain: VS Code + MSYS2 UCRT64 GCC + FreeGLUT

Student Name:	Ebron, Nicole Anne V.
Section:	TN35
Professor:	Dr. Hadji J. Tejuco

GitHub Repository: https://github.com/nicoleanneebron-art/Ebron-Nicole-Anne-V.

ChatGPT Conversation: https://chatgpt.com/share/e/6acb86d9-1b24-83ec-9a3b-957c0e433cb7

A. Prepare Your Project Proposal

Project Title: Run Chicken!

Problem / Idea:
Run Chicken! is a 2D endless runner mini-game featuring an escaped chicken navigating through various obstacles in an attempt to survive for as long as possible. Inspired by popular games such as *Chrome Dinosaur Game*, *Zombie Tsunami*, and *Temple Run*, the project combines simple controls, continuous movement, and progressively challenging gameplay to create an engaging arcade experience.

Purpose:
The project aims to develop an entertaining and interactive graphical application using C++ and OpenGL/FreeGLUT. It demonstrates the practical application of computer graphics concepts, including geometric primitives, colors, animations, keyboard and mouse interactions, and collision detection.

Target User:
Run Chicken! is designed for players of all ages, particularly those who enjoy casual arcade games. Its straightforward controls and simple gameplay mechanics make it accessible to beginners while providing an enjoyable challenge for more experienced players.

Objects:
The game features a chicken as the main playable character, accompanied by a scrolling environment consisting of mountain scenery, ground platforms, clouds, and other background elements. Additional objects include obstacles such as fences and flying birds, as well as collectible items, interactive menu buttons, and on-screen text displays.

User Interaction:
Players interact with the game through keyboard and mouse controls:

- **Space Bar:** Makes the chicken jump to avoid obstacles.
- **P:** Pauses or resumes gameplay.
- **R:** Restarts the game.
- **ESC:** Exits the application.
- **Mouse Click:** Allows players to navigate and select options from the game menus.

Expected Output:
A fully functional 2D endless runner game developed using C++ and OpenGL/FreeGLUT. The application is expected to feature animated characters, continuously scrolling scenery, interactive controls, obstacle collision detection, a scoring system, and functional game menus. The finished project should provide a smooth and engaging gameplay experience while demonstrating fundamental computer graphics and programming concepts.

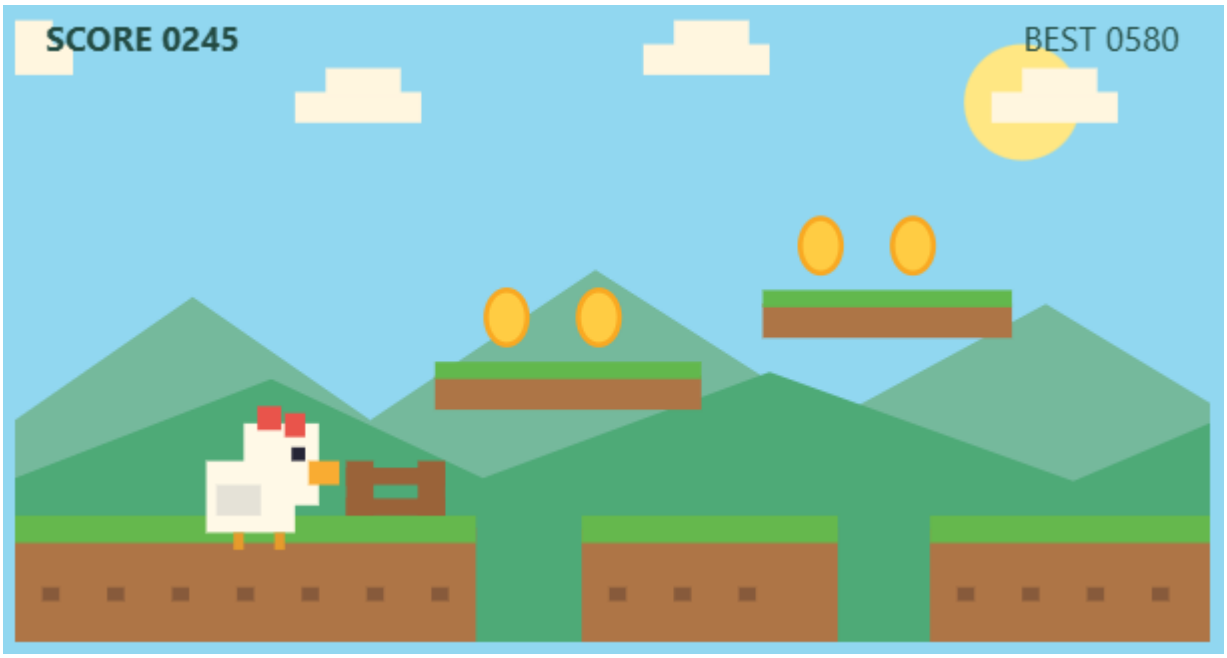
B. Brainstorming Summary / Initial Ideas

The initial concept for this project was a farming simulation game in which players would grow oversized vegetables, particularly squashes and pumpkins. This idea was inspired by Sarah Witty, a pumpkin grower and competitor whose content I encountered on Facebook. However, I encountered difficulties in developing engaging gameplay mechanics and maintaining the fast-paced experience I wanted to achieve.

As an alternative, I explored a maze-based arcade game inspired by *Tomb of the Mask*, where players would navigate through obstacles while racing against time. However, designing a functional and engaging maze using predefined patterns proved challenging. Creating well-structured maps would require extensive planning and testing, making the concept less practical within the project's development timeframe.

Considering these challenges, I decided to pursue my third concept, *Run Chicken!*, a 2D endless runner game. This approach offered a more manageable development process while still allowing me to incorporate interactive gameplay, animations, and various computer graphics concepts required for the project.

C. Sketch / Initial Design



D. List of Objects

Objects and Graphical Elements

Run Chicken! incorporates various pixel-styled graphical objects that establish its farm-inspired environment, support gameplay mechanics, and provide interactive user interfaces. These objects are categorized according to their functions and visual characteristics.

1. Main Character

- **Chicken:** The primary playable character designed using pixelated geometric shapes. It features running and jumping animations that respond to player input and gameplay conditions.

2. Background Scenery

- **Mountain Ranges:** Multiple layers of mountains in varying shades of green, creating depth within the scrolling landscape.
- **Pine Trees:** Repeated triangular tree structures arranged across the foreground and background to establish a forest-like environment.
- **Clouds:** Pixelated, light-colored clouds distributed across the sky to enhance the outdoor atmosphere.
- **Sun:** A large, circular pixel-styled sun with yellow and orange layers, positioned in the upper portion of the scene.
- **Decorative Birds:** Small, dark-colored bird silhouettes scattered throughout the sky to make the environment appear livelier.
- **Sky:** A light blue background that establishes the daytime setting.
- **Atmospheric Details:** Small horizontal streaks distributed across the sky, adding subtle visual texture.

3. Environment and Structures

- **Chicken Coop:** A large red barn-style structure featured on the initial screen. It consists of a triangular roof, rectangular walls, windows, and double doors, establishing the starting location of the chicken.
- **Ground and Terrain:** A horizontal grassy surface supported by a brown soil layer with small rectangular patterns, serving as the main running path.
- **Ground Platforms:** Elevated or separated surfaces used for platforming sections, where implemented.

4. Gameplay Obstacles

- **Wooden Fences:** Small wooden barriers that the chicken must jump over to avoid collisions.
- **Hay Bales:** Farm-themed obstacles that introduce additional challenges during gameplay.
- **Wooden Crates:** Box-shaped obstacles that require players to time their jumps appropriately.
- **Flying Birds:** Moving aerial obstacles that travel horizontally at a fixed height, requiring players to avoid collisions. Unlike decorative birds in the background, flying bird obstacles directly affect gameplay through collision detection.

5. Collectible Objects

- **Golden Eggs:** Bright yellow, circular collectibles positioned along the running path or near obstacles. Collecting these contributes to the player's score or collectible count.

6. Menu and Interface Elements

- **Game Title:** The pixel-styled *RUN CHICKEN!* text displayed prominently on the initial screen.
- **Wooden Plank Buttons:** Rectangular, wood-textured menu buttons with dark outlines and pixelated labels. These provide a consistent visual theme across the game's interfaces.
- **Start Menu:** Contains the Play, Rankings, Credits, and Exit buttons.
- **Pause/Resume Menu:** Provides options for resuming gameplay, restarting the run, or exiting the game.
- **Game Over Menu:** Displays the player's results and provides a Run Again option.
- **Rankings Screen:** An interface intended to display player scores or rankings.
- **Credits Screen:** An interface containing project or developer information.

7. Heads-Up Display (HUD)

The HUD is positioned at the top of the gameplay screen and consists of dark rectangular panels with colored upper borders and pixel-styled numerical displays.

- **Score:** Displays the player's accumulated points during the current run.
- **Golden Eggs:** Tracks the number of golden eggs collected.
- **Chick Lives:** Displays the player's remaining lives or available chances.
- **Run Speed:** Indicates the current movement speed of the game.
- **Best Score:** Displays the highest recorded score.

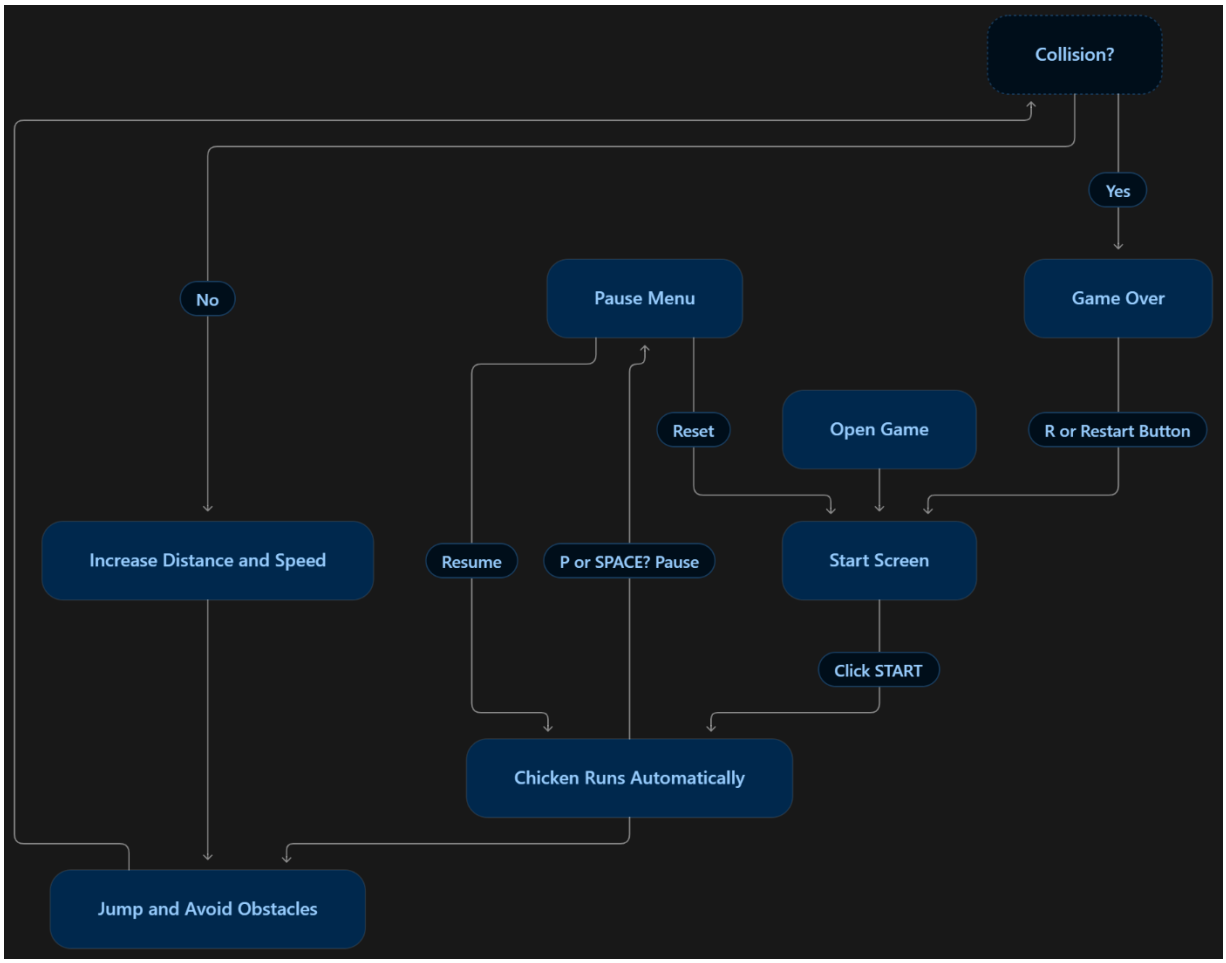
8. Text, Animations, and Visual Effects

- **Pixel-Styled Typography:** Custom block-like lettering used for the title, menu labels, HUD information, and other on-screen text.
- **Chicken Animations:** Frame-based or position-based visual changes that represent running, jumping, and other character actions.
- **Scrolling Scenery:** Horizontal movement of environmental objects that creates the illusion of continuous forward motion.
- **Menu Transitions:** Visual changes between the start screen, gameplay, pause menu, and game-over screen.
- **Color Variations:** Different color shades applied to environmental objects to establish visual depth and maintain the retro aesthetic.

E. OpenGL Primitives Used

The game uses several OpenGL primitives to create its 2D pixel-art visuals. **GL_QUADS** is mainly used for rectangles such as platforms, HUD panels, clouds, menu planks, chicken body parts, and individual pixel-font blocks**. GL_TRIANGLES** creates angular objects such as mountains, pine trees, and tree branches. **GL_TRIANGLE_FAN** forms circular shapes like the sun by connecting a center vertex to multiple points around its edge. The sky gradient also uses a four-vertex quad with different colors so OpenGL can blend the colors smoothly. These simple primitives are combined to construct the game’s characters, environment, menus, and interface.

F. Program Flow



G. Controls

- **Space / W** — Jump; press again in the air to double-jump.
- **P** — Pause or resume the game.
- **R** — Restart the run.
- **Esc** — Exit the game.
- **Mouse click** — Select menu buttons or jump during gameplay.

H. Explanation of Functions

The game is organized into functions, with each function handling a specific responsibility. Some functions draw objects such as rectangles, triangles, circles, trees, the chicken, menus, and the HUD. Other functions control gameplay logic, including jumping, collision detection, level generation, scoring, follower lives, and game-state changes. Separating these responsibilities allows the same drawing and gameplay operations to be reused throughout the program.

I. Explanation of Conditions

Conditions are used to make decisions during gameplay. They determine the current game state, such as whether the game is on the start screen, playing, paused, or over. Conditions also check whether the player is grounded, jumping, colliding with an obstacle, collecting an egg, falling into a

gap, or still has chick lives remaining. For example, the game only updates player movement when the state is ***PLAYING***, and the game only ends when the player has no remaining protection or falls below the level.

J. Explanation of Loops

Loops are used to repeat operations efficiently. The game loop repeatedly updates the game and redraws the screen using the timer and display callbacks. ***for*** loops draw repeated background elements such as clouds, trees, mountains, wind marks, and menu supports. Loops also process vectors of platforms, obstacles, eggs, birds, and chicks so that every object can be updated, drawn, or checked for collisions. This avoids writing separate code for every individual object.

K. Explanation of Keyboard Callback

The keyboard callback is handled by the ***keyboard()*** function. It responds to key presses from the player. The Spacebar, **W**, and **w** are used for jumping or starting the game. The **P** key pauses or resumes gameplay, while **R** restarts the game. The Escape key exits the application. The function also checks the current game state so that the same key can perform different actions depending on whether the game is playing, paused, or on the game-over screen.

L. Explanation of Mouse Callback

The mouse callback is handled by the ***mouse()*** function. It converts the mouse's window coordinates into the game's virtual coordinate system so that clicks remain accurate in fullscreen mode. The callback detects clicks on menu planks such as ***PLAY, CONTINUE, RESUME, RESTART, RUN AGAIN, and EXIT***. It also starts the plank press animation before performing the selected action. During gameplay, clicking in the designated control area can trigger a jump.

M. Explanation of Text Callback

The project does not use a traditional text callback. Text is rendered through functions such as ***textColor(), pixelGlyph(), pixelText(), pixelCenteredText(), and pixelCenteredTextSolid()***. Standard bitmap text is used for some HUD elements, while the menus use a custom 5×7 pixel-font system. Each supported character is stored as a pattern of 1 and 0 values. The ***pixelText()*** function converts the filled values into small square shapes, allowing menu labels, scores, and game-over text to match the pixel-art style.

N. Explanation of Vertex Arrays

There are two different kinds of arrays in the project:

1. Arrays used for OpenGL vertex geometry.
2. Arrays or vectors used to store game objects and pixel-font data.

A. What data is stored in the arrays?

Vertex Arrays

In the quads( ) function:

```
GLfloat vertices[] = {
      x, y,
      x + w, y,
      x + w, y + h,
```

```
      x, y + h
};
```

This array stores four 2D vertex positions. Each pair of values represents one vertex; these are the four corners of a rectangle.

The triangle( ) function stores three vertex positions:

```
GLfloat vertices[] = {
      x1, y1,
      x2, y2,
      x3, y3
};
```

The circle() function stores:

- The center point.
- Multiple points around the circle's circumference.

These points are stored in a std::vector<GLfloat>.

Game-object Vectors

The game also stores objects in vectors:

```
std::vector<Obstacle> obstacles;
std::vector<Platform> platforms;
std::vector<Egg> eggs;
std::vector<Bird> birds;
std::vector<LifeChicken> lifeChickens;
```

These arrays do not directly store vertices. They store gameplay data such as:

- Position
- Width and height
- Obstacle type
- Bird movement phase
- Chick falling state

This allows the game to update and draw many objects using loops.

Pixel-font arrays

The custom pixel font also uses arrays. For example, each letter is stored as a 5×7 pattern:

```
static const char* A[] = {
      "01110",
      "10001",
      "10001",
      "11111",
      "10001",
      "10001",
      "10001"
};
```

Each 1 represents a filled pixel, while each 0 represents empty space.

B. Why use arrays?

Arrays are useful because they group related data together.

For vertex geometry, the array allows OpenGL to receive multiple coordinates as one organized block of data.

For gameplay objects, vectors allow the game to:

- Store many objects.
- Loop through them.
- Move them.
- Check collisions.
- Remove them when they leave the screen.
- Generate new objects dynamically.

For the pixel font, arrays allow each letter to be described compactly and reused whenever that letter appears.

C. What does each vertex represent?

A vertex represents a 2D position in the game world.

For a rectangle:

Vertex 1: bottom-left corner
Vertex 2: bottom-right corner
Vertex 3: top-right corner
Vertex 4: top-left corner

For a triangle:

Vertex 1: first corner
Vertex 2: second corner
Vertex 3: third corner

For the circle:

Vertex 1: center of the circle
Remaining vertices: points around the circumference

D. What OpenGL primitive is being rendered?

The project uses several OpenGL primitives.

GL_QUADS

Used for rectangles:

```
glDrawArrays(GL_QUADS, 0, 4);
```

This is used for:

- Platforms
- HUD panels
- Clouds
- Chicken body parts
- Menu planks
- Pixel-font blocks

GL_TRIANGLES

Used for triangles:

glDrawArrays(GL_TRIANGLES, 0, 3);

This is used for:

- Mountains
- Pine trees
- Branch layers
- Angular decorations

GL_TRIANGLE_FAN

Used for circles:

glDrawArrays(GL_TRIANGLE_FAN, 0, segments + 2);

This is used for:

- The sun
- Circular decorative shapes

GL_QUADS with glBegin

The sky gradient uses four vertices with different colors:

glBegin(GL_QUADS);

OpenGL interpolates the colors between the vertices to create the smooth gradient.

E. Why is this better than repeatedly specifying the same information?

Using arrays makes the code:

- More organized.
- Easier to reuse.
- Easier to animate.
- Less repetitive.
- Less likely to contain inconsistent coordinates.
- Easier to update when the size or position changes.

O. Important Code Sections

The most important part of the game is the obstacle-pattern and gameplay logic. The addPattern() function creates repeating level sections containing platforms, gaps, fences, hay obstacles, eggs, and birds. Different patterns are selected using pattern % 4, allowing the game to continuously generate varied sections instead of using one fixed level. The generateWorld() function keeps adding new patterns ahead of the player as the existing level moves off-screen.

The updateGame() function controls the main gameplay behavior. It increases the game speed, moves platforms and obstacles toward the player, updates eggs and birds, checks collisions, updates chick followers, and removes objects that have left the screen. It also calls updatePlayer() to handle gravity, jumping, falling, and platform landing. Together, these functions create the endless-runner structure of the game.

The game-state logic is also important. The states START_SCREEN, PLAYING, PAUSED, and GAME_OVER determine which systems are active and which screen is displayed. This allows the same program to support the opening barn transition, normal gameplay, the pause menu, and the game-over screen.

P. Creative Features

The game includes several creative features that give it its own identity. These include animated clouds, moving mountains, a layered pixel-art forest, animated birds, chicken wing movement, chick followers, feather particles, and a smooth chicken jump from the barn into gameplay. The game also includes collision detection, a score system based on distance and collected eggs, increasing game speed, gravity-based jumping, double-jumping, and falling pit hazards.

The interface includes multiple screens, including the initial start screen, pause screen, and game-over screen. The wooden plank menu system gives the game a distinctive barn-themed style, and the interactive buttons visibly depress when clicked. Other features include procedural object spawning, multiple obstacle patterns, chick-based extra lives, bird obstacles, animated HUD elements, full screen support, and gradual difficulty increase. Together, these features combine simple programming techniques into a complete playable game with a consistent visual theme.

Q. Screen Shots



Figure 1. Initial Screen

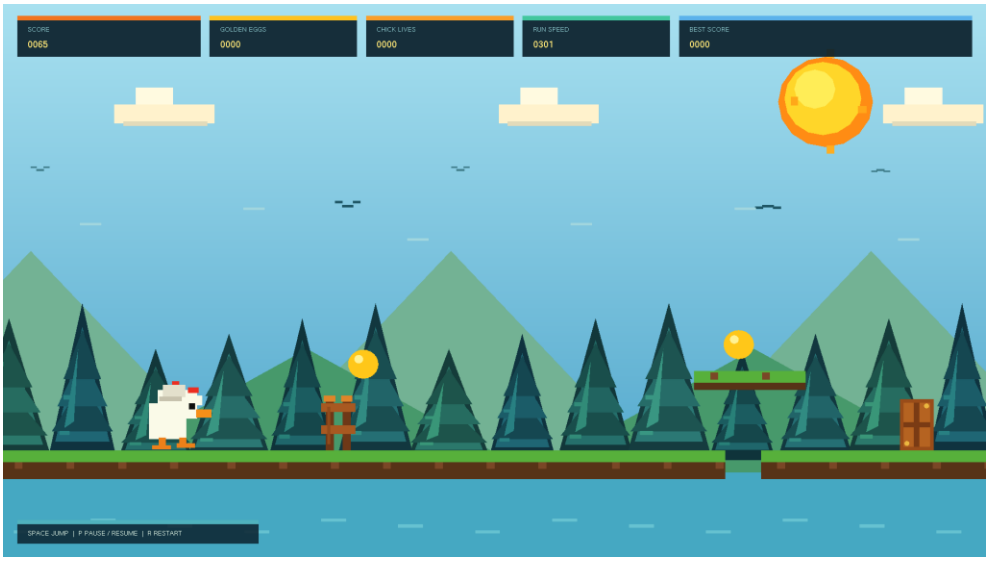


Figure 2. Main Application



Figure 3. Keyboard Interaction (Space Bar or 'W' Key)



Figure 4. Mouse Interaction (Click to interact with Buttons)

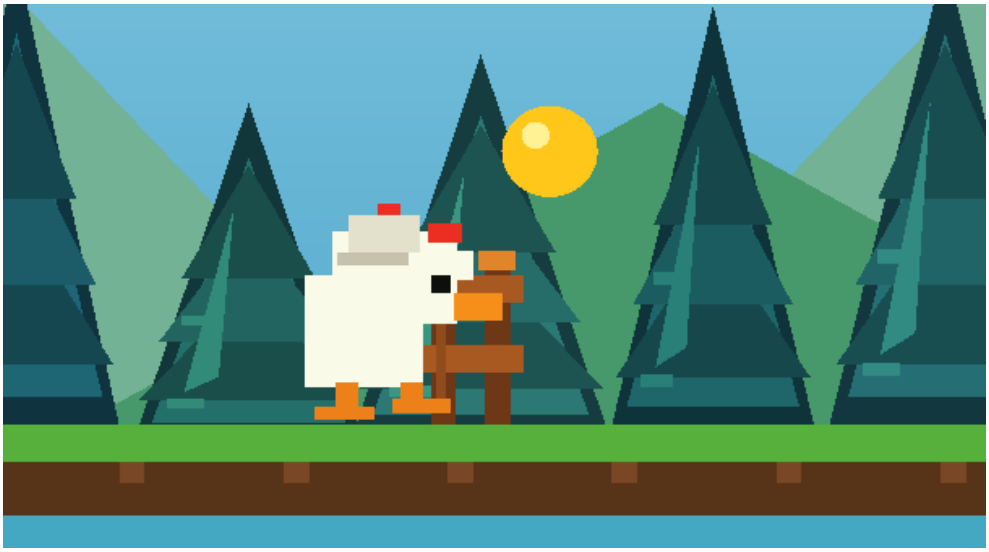


Figure 5. Collision Detection



Figure 6.1 Initial Home Screen



Figure 6.2 Pause/Resume Screen



Figure 6.3 Game Over Screen



Figure 7. Heads Up Display



Figure 8. Controls

R. 10+ Test Cases

Test	Action/Input	Expected Result	Actual Result	Status
1	Launch the game	Game opens in full screen on the start screen	Game opens full screen with the animated background	PASS
2	Wait on the start screen	Clouds and birds continue moving	Clouds drift and birds' flap across the sky	PASS
3	Click PLAY	Barn intro begins and the chicken jumps out	Chicken exits the barn and gameplay starts	PASS
4	Press Space	Chicken jumps	Chicken jumps with wing animation	PASS
5	Press Space twice	Chicken performs a double jump	Second jump is allowed while airborne	PASS
6	Press P during gameplay	Game pauses	Gameplay stops and the pause menu appears	PASS
7	Click RESUME	Game continues	Resume plank animates and gameplay continues	PASS
8	Press P while paused	Game resumes	Game returns to the running state	PASS
9	Press R	Game restarts	World, score, eggs, and player state reset	PASS

10	Click RESTART	Game begins a new intro sequence	Restart plank depresses before the new run begins	PASS
11	Run into a fence with chicks	One chick is removed instead of ending the game	One follower falls and the run continues	PASS
12	Let a bird hit the chicken with chicks	Only one chick is caught	One chick is removed and the bird collision is handled	PASS
13	Run into an obstacle with no chick protection	Run ends	Game-over screen appears	PASS
14	Fall into a pit gap	Chicken and followers fall	The player falls and the run ends if no protection remains	PASS
15	Collect an egg	Egg disappears and score increases	Egg is collected and the score updates	PASS
16	Collect five eggs	A chick follower is added	An additional chick appears behind the player	PASS
17	Continue running	Obstacle patterns continue spawning	New platforms, gaps, eggs, and obstacles appear ahead	PASS
18	Continue running for a long period	Difficulty gradually increases	Game speed increases and birds appear more often	PASS
19	Observe the background loop	Mountains, trees, clouds, birds, and wind marks loop smoothly	Background layers repeat without visible gaps	REVISED
20	View the forest background	Trees appear tightly packed and do not hide the mountains	Tree spacing and layered design were adjusted after review	REVISED
21	View the game-over text	“GAME OVER,” score, and best score display correctly	Missing pixel letters were identified and added	REVISED
22	Click a wooden menu plank	Plank visibly depresses before the action occurs	Click animation was added with a short, delayed action	PASS
23	Press an invalid key	No unintended game action occurs	Game state remains unchanged	PASS
24	Press Escape	Game closes	Application terminates	PASS
25	Resize or run full screen	Menus remain centered and usable	Fullscreen rendering fills the window and mouse input remains aligned	PASS

S. How ChatGPT Was Used

ChatGPT served as a planning, programming, and documentation assistant throughout the development of *Run Chicken!*. During the initial planning stage, it helped brainstorm and evaluate different game concepts, identify potential development challenges, and organize the project's objectives, gameplay mechanics, visual design, and implementation plan. This included refining the concept into a 2D endless runner and outlining the development process into manageable stages.

During implementation, ChatGPT assisted in developing and improving various game features, including character movements, animations, interactive menus, the Heads-Up Display (HUD), scrolling backgrounds, collision detection, obstacle patterns, and full screen functionality. It also provided guidance on applying C++ and OpenGL/FreeGLUT concepts, such as geometric primitives, vertex arrays, loops, functions, and keyboard and mouse interactions.

Additionally, ChatGPT was used for debugging, code refinement, exploring testing approaches, and improving the game's pixel-art presentation. It also assisted in organizing and polishing the project documentation. Throughout development, the generated suggestions and solutions were reviewed, tested, and adjusted based on the project's requirements and the desired gameplay experience.

T. Challenge ChatGPT

AI SUGGESTIONS

AI Suggestion:	Use standard bitmap fonts for the menus.
My Decision:	Rejected.
Reason:	The letters looked too thin and generic, so I requested a custom block-pixel font with stronger outlines.
AI Suggestion:	Use the existing background loop periods.
My Decision:	Rejected.
Reason:	Some elements could visibly snap or leave gaps when looping, so the repeat periods were aligned with each element's spacing.
AI Suggestion:	Use only tiny pit gaps.
My Decision:	Revised.
Reason:	The gaps were difficult to notice, so I requested slightly wider pits that could punish careless jumping while remaining fair.
AI Suggestion:	Keep the idle background static.
My Decision:	Rejected.
Reason:	I wanted the start screen to feel alive, so the clouds now drift and distant birds' flap and move during the idle scene.
AI Suggestion:	Remove unused legacy menu and text-rendering functions.
My Decision:	Accepted.
Reason:	The cleanup preserved the game's behavior while removing code that was no longer used by the current menu system.
AI Suggestion:	Use comments to document every active function.
My Decision:	Accepted.
Reason:	I wanted the code to be easier to explain and document without changing the existing logic or comments.

U. Challenge ChatGPT

1. What was the most difficult part?

One of the most challenging parts was creating a custom pixel-styled text format that would complement the game's retro aesthetic. Unlike standard text rendering, this required additional functions, patterns, and loops to construct each character using geometric shapes. There were also issues on improving the smooth and seamless transitions of the animations especially the starting animation once the game began.

2. What bug did you encounter?

One of the main issues I encountered involved the obstacle generation logic, particularly maintaining fair and consistent spacing between obstacles. Some generated patterns placed obstacles too close together, making certain sections unnecessarily difficult and affecting the overall gameplay experience.

3. How did you fix it?

I initially addressed the issue by increasing the spacing between obstacles to give players sufficient time to react. Since the game's running speed gradually increases, the difficulty would naturally progress without requiring obstacles to appear too close together. I also reduced the frequency of pitfalls and adjusted the corresponding values and coordinates to create a more balanced and enjoyable gameplay experience.

4. What did ChatGPT help you understand?

ChatGPT helped me better understand the structure and logic behind the program, particularly how animations work and how different functions interact. Its explanations and code breakdowns made it easier to follow the implementation and understand the purpose of individual sections of the code.

5. What did ChatGPT suggest that you change?

There were relatively few major changes suggested, as the overall game concept and its primary mechanics had already been established. Most of the assistance focused on refining the existing ideas, explaining their implementation, and addressing technical challenges encountered during development.

6. What part of the program did you personally improve?

I focused on improving the game's animations and transitions to create a smoother and more engaging experience. Although I received assistance with the implementation and did not write every part of the code entirely from scratch, the decision to incorporate additional animations came from my own ideas. Rather than relying mainly on static graphical elements, I wanted the game to feel more dynamic, particularly during transitions between different scenes.

7. If you had another week, what feature would you add?

Given another week, I would prioritize implementing a file-saving system to preserve the player's best scores even after closing the application, there’s fun in a little competition and if all possible, a two-player mode sounds fun. I would also like to introduce a day-and-night cycle with gradual background color transitions, along with additional creative levels and environmental designs to make the gameplay more visually diverse and engaging. For the technical side, I’d like to reorganize the code into separate files to better navigate and improve in the future.